Robust Reinforcement Learning Control of a Non-linear Quadrotor Under Stochastic Disturbances

Henry Edward Fairfax Rochester

Research Project Proposal
Applied Mathematics Project MAM3055Z

Department of Mathematics and Applied Mathematics
University of Cape Town

Supervisor: Dr Malcolm Hillebrand

February 22, 2026

Abstract

This project investigates the robustness and stability of a learned non-linear reinforcement learning (RL) controller for quadrotor dynamics under stochastic disturbances. We compare a model-free Proximal Policy Optimisation (PPO) controller against classical control baselines: PID control and a Linear Quadratic Regulator (LQR) controller. The quadrotor is modelled as a non-linear dynamical system derived from Newton's second law as well as the Newton-Euler rigid body equations. The robustness of the controller is evaluated under observation noise, wind disturbances, and drone-mass variations. Moreover, the performance metrics would include tracking error, recovery time, failure rate, and energy efficiency.

1 Introduction

Quadrotor drones are non-linear dynamical systems with coupled translational (up/down, left/right, forward/backward) and rotational (roll, pitch and yaw) dynamics. Their motion is governed by Newton's Second Law and the Newton-Euler equations for rigid body rotation [4]. Classical quadrotor modelling and control have shown PID controllers [2, 3], and optimal linear control via LQR as standard baselines for quadrotor performance [5].

Modern drones typically rely on PID controllers due to simplicity and ease of implementation. However, these controllers require heuristic tuning and do not explicitly optimise performance criteria, as there is no model present within PID controllers.

Recent advances in reinforcement learning (RL), particularly policy gradient methods such as PPO, allow direct optimisation of non-linear control policies without requiring an explicit model of the system. This project investigates whether a learned non-linear RL controller can outperform classical PID and LQR controllers in terms of robustness and stability.

(I understand that I would need to go into a bit of detail in the actual presentation of the paper, to give background as to what PID, LQR controllers are, as well as what the PPO policy gradient method is and how it works).

2 Quadrotor Dynamics

2.1 State Representation

Let the state vector be given by:

$$\vec{x} = \begin{bmatrix} \vec{p} \\ \vec{v} \\ \vec{\eta} \\ \vec{\omega} \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \\ \phi \\ \theta \\ \psi \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \in \mathbb{R}^{12}.$$

The components are defined as follows:

- $\vec{p} = (x, y, z)^\top \in \mathbb{R}^3$ is the position of the quadrotor's centre of mass in the inertial reference frame, we refer to this as the 'world' frame;
- $\vec{v} = (v_x, v_y, v_z)^\top \in \mathbb{R}^3$ is the linear velocity in the inertial reference frame;
- $\vec{\eta} = (\phi, \theta, \psi)^\top \in \mathbb{R}^3$ represents the Euler angles (which describe the orientation of the drone body's reference frame relative to the inertial reference frame...I will superimpose an image here to give the reader an easier way to understand):
  - ϕ - roll angle,
  - θ - pitch angle,
  - ψ - yaw angle.
- $\vec{\omega} = (\omega_x, \omega_y, \omega_z)^\top \in \mathbb{R}^3$ is the angular velocity vector expressed in the drone body's reference frame, where:
  - ω_x is angular velocity about the body's x-axis,
  - ω_y is angular velocity about the body's y-axis,
  - ω_z is angular velocity about the body's z-axis.

The control input is the rotor thrust vector:

$$\vec{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} \in \mathbb{R}^4,$$

where u_i represents the thrust generated by rotor i.

2.2 Translational Dynamics

From Newton's Second Law, we have:

$$m\ddot{\vec{p}} = m\vec{g} + \mathbf{R}(\vec{\eta})\vec{F}_T,$$

where:

- m is the mass of the drone,
- $\vec{g} = (0, 0, -g)^\top$ is the gravitational acceleration vector in the inertial reference frame,
- $\mathbf{R}(\vec{\eta}) \in SO(3)$ is the rotation matrix parameterised by the Euler angles $\vec{\eta} = (\phi, \theta, \psi)^\top$,
- $\vec{F}_T$ is the total thrust vector expressed in the drone body's reference frame.

2.3 Rotational Dynamics

Using the Newton-Euler rigid body dynamics [4], the rotational motion of the quadrotor is governed as follows:

$$\mathbf{I}\dot{\vec{\omega}} + \vec{\omega} \times (\mathbf{I}\vec{\omega}) = \vec{\tau},$$

where:

- $\vec{\omega} = (\omega_x, \omega_y, \omega_z)^\top \in \mathbb{R}^3$ is the angular velocity vector expressed in the drone body's reference frame;
- $\mathbf{I} \in \mathbb{R}^{3\times 3}$ is the inertia matrix of the quadrotor about its centre of mass;
- $\vec{\tau} = (\tau_x, \tau_y, \tau_z)^\top \in \mathbb{R}^3$ is the control torque generated by differential rotor thrust.

(I still need to go through and study these equations in more detail to understand how each component shall affect the dynamics of the system)

Combining translational and rotational dynamics, the full non-linear system can be written compactly as the following system:

$$\dot{\vec{x}} = f(\vec{x}, \vec{u}),$$

where $\vec{x} \in \mathbb{R}^{12}$ is the state vector and $\vec{u} \in \mathbb{R}^4$ is the rotor thrust input vector.

3 Control Baselines

3.1 PID Controller

As one of our baselines, we consider a classical Proportional-Integral-Derivative (PID) feedback controller. Let $\vec{x}^*$ denote the desired hover (or target) state. We define the tracking error as follows:

$$\vec{e}(t) = \vec{x}(t) - \vec{x}^*.$$

The PID control law is then given by the following equation (which encompasses a proportional component, integral and derivative):

$$\vec{u}(t) = -K_P\vec{e}(t) - K_I \int_0^t \vec{e}(s)\, ds - K_D\dot{\vec{e}}(t),$$

where K_P, K_I, K_D are proportional, integral, and derivative matrices, respectively.

The proportional term proportionally penalises deviation from the desired state; the integral term accumulates past error to eliminate steady-state bias (i.e. if the quadrotor reaches a height just below the desired one and remains in a steady position due to a constant wind force blowing downward), and the derivative term acts as a damping term, allowing for anticipation of future behaviour (i.e. when approaching desired target, derivative will become less and less and thus contribution will become smaller).

Although PID control is widely used in commercial quadrotors due to its simplicity and ease of implementation, it does not rely on an explicit model of the system dynamics and requires heuristic tuning of the respective matrix parameters. Consequently, its robustness under non-linear dynamics and stochastic disturbances may be limited compared to model-based (LQR) or learned control (RL PPO) strategies.

3.2 Linear Quadratic Regulator (LQR)

As a model-based baseline, we consider the Linear Quadratic Regulator (LQR). The idea is to linearise the non-linear quadrotor dynamics around a hover equilibrium state, yielding a linear system of the following form:

$$\dot{\vec{x}} = A\vec{x} + B\vec{u},$$

where A and B are the linearised Jacobi matrices of the non-linear dynamics evaluated at the equilibrium point.

LQR seeks a state-feedback control law of the following form:

$$\vec{u} = -K\vec{x},$$

where the "gain" matrix K is chosen to minimise the quadratic cost function:

$$J = \int_0^\infty \left(\vec{x}^\top Q\vec{x} + \vec{u}^\top R\vec{u}\right)\, dt.$$

Here, Q, a positive matrix, penalises deviation from the state equilibrium, and R, also a positive matrix, penalises how much thrust is input (which makes sense as we would have limited battery power). The optimal gain matrix is given by:

$$K = R^{-1}B^\top P,$$

where P is the unique solution of the continuous-time Riccati equation:

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0.$$

Intuitively, as I understand it, the LQR controller balances the trade-off between importance of reaching target state and importance of minimisation of thrust input. Unlike PID control, LQR explicitly incorporates a model of the system's dynamics

At this stage, this subsection serves as a pre-liminary outline of how the LQR controller would be implemented. Further reading is required to fully understand the derivation of the Riccati equation, the cost function and an understanding of the optimal gain matrix. A deeper study of the LQR framework will be undertaken prior to implementation.

3.3 Control Objective

The primary objective of this project is to design and evaluate controllers that enable the quadrotor to autonomously reach and stabilise (hover for a specific period of time - TBD) at a specified target position in three-dimensional space.

Let the desired target state be denoted as follows:

$$\vec{x}^* = \begin{bmatrix} \vec{p}^* \\ \vec{0} \\ \vec{\eta}^* \\ \vec{0} \end{bmatrix},$$

where $\vec{p}^* \in \mathbb{R}^3$ represents the desired target position, $\vec{\eta}^*$ corresponds to the desired orientation (typically level hover), and the zero vectors indicate the requirement of zero linear and angular velocity at equilibrium.

The control objective is therefore to design a feedback law $\vec{u}(t)$ such that:

$$\lim_{t \to \infty} \vec{x}(t) = \vec{x}^*,$$

while maintaining stability and robustness (under noisy observations, windy conditions and varying payload...the metrics of how we measure this is discussed below) under stochastic disturbances.

We compare three control strategies for achieving this objective:

1. A classical PID controller (model-free baseline),
2. A Linear Quadratic Regulator (model-based optimal linear control),
3. A learned non-linear policy trained using the Proximal Policy Optimisation (PPO) RL algorithm.

Each controller is evaluated based on its ability to:

- drive the quadrotor to the desired target position;
- stabilise at hover;

- recover from disturbances;
- minimise tracking error and control effort.

The central aim of the project is to effectively determine whether a learned non-linear reinforcement learning controller can outperform classical PID and LQR approaches in terms of robustness, stability, and energy efficiency when operating under realistic stochastic disturbances.

4 Reinforcement Learning Controller

Here is where it becomes interesting, we now consider a model-free non-linear controller trained using Proximal Policy Optimisation (PPO).

The control policy is represented by a neural network parameterised by θ:

$$\vec{u}(t) = \pi_\theta(\vec{x}(t)),$$

where $\pi_\theta : \mathbb{R}^{12} \to \mathbb{R}^4$ maps the current state of the quadrotor to rotor thrust commands.

The control problem is formulated as a Markov Decision Process (MDP), in which:

- the state is given by the quadrotor state vector $\vec{x}$;
- the action corresponds to the control input $\vec{u}$;
- the reward function (TBD) would penalise tracking error and control effort;
- the dynamics evolve according to the non-linear system $\dot{\vec{x}} = f(\vec{x}, \vec{u})$;
- (the discount factor would be a hyperparameter that we would need to tune).

For some context, PPO is a policy-gradient method that seeks to maximise the expected cumulative reward by iteratively updating the policy parameters θ using a clip objective function. (I shall explain this in more detail in the actual project write-up). Unlike PID or LQR, this approach does not require explicit knowledge of the system matrices or linearisation around an equilibrium point.

(At the current stage of this project, I am actively studying and implementing the PPO algorithm, drawing primarily from the reinforcement learning foundations presented in [1].)

4.1 Research Question

How does a learned non-linear RL controller compare to classical PID and LQR control in terms of stability and robustness under stochastic disturbances in quadrotor dynamics?

5 Stochastic Disturbances

In order to evaluate robustness, we introduce controlled stochastic perturbations into the system. These disturbances are designed to emulate realistic sources of uncertainty encountered in physical systems, including sensor noise, environmental forces, and parameter mismatch.

5.1 Observation Noise

To model imperfect sensing, we assume that the controller does not observe the true state $\vec{x}_t$ directly, but instead receives a noisy measurement:

$$\tilde{\vec{x}}_t = \vec{x}_t + \vec{\epsilon}_t, \qquad \vec{\epsilon}_t \sim \mathcal{N}(\vec{0}, \sigma^2 I).$$

This captures the effect of sensor inaccuracies and electronic interference, and allows us to assess the robust sensitivity of each controller to noisy state information.

5.2 Wind Disturbance

To represent environmental perturbations such as wind, we introduce an additive stochastic forcing term into the translational ODE:

$$m\ddot{\vec{p}} = m\vec{g} + R(\vec{\eta})\vec{F}_T + \vec{w}_t, \qquad \vec{w}_t \sim \mathcal{N}(\vec{0}, \sigma_w^2 I).$$

This disturbance directly affects the acceleration of the quadrotor and tests the controller's ability to recover from external forces acting on the drone.

5.3 Mass Variation (Payload Shift)

To evaluate robustness to modelling error and payload variability, we train the controller under a nominal mass m_{train} and evaluate performance under a different mass, such as:

$$m_{\text{test}} \neq m_{\text{train}}.$$

This simulates the practical scenario in which a quadrotor carries different payloads, thereby altering its inertia. Controllers that rely heavily on accurate modelling may degrade under such parameter mismatch.

6 Evaluation Metrics

To assess both stability and robustness of the proposed controllers, we utilise the following quantitative metrics. Stability is interpreted in the practical sense of convergence toward the desired equilibrium state $\vec{x}^*$, while robustness refers to performance under stochastic disturbances and parameter variation.

6.1 Tracking Error

To measure convergence toward the target position $\vec{p}^*$, we define the average tracking error thusly:

$$E_{\text{track}} = \frac{1}{T} \int_0^T \|\vec{p}(t) - \vec{p}^*\| \, dt.$$

A stabilising controller should drive this quantity toward zero as T increases.

6.2 Recovery Time

To quantify practical stability and the drone's ability to return to its desired state after being perturbed, let t_0 denote the time at which a disturbance is introduced. We define the recovery time as:

$$T_{\text{recover}} = \min \{t > t_0 : \ \|\vec{x}(t) - \vec{x}^*\| < \varepsilon\},$$

where ε is a prescribed tolerance (TBD). A smaller recovery time indicates stronger stabilising behaviour under perturbation.

6.3 Failure Rate

We evaluate stability in a binary sense (and as a percentage of total episodes) by defining failure conditions such as (we can define more too):

$$z(t) \leq 0 \quad \text{and} \quad \|\vec{v}(t)\| > v_{\text{crash}},$$

corresponding to ground collision with excessive velocity. The failure rate is defined as the percentage of simulation episodes in which such failure occurs.

6.4 Energy Efficiency

To assess thrust efficiency and smoothness, we define the total actuation intensity as:

$$E_{\text{energy}} = \int_0^T \|\vec{u}(t)\|^2 \, dt,$$

which penalises aggressive thrust usage and large control spikes. Controllers that achieve stability with lower energy expenditure are considered more efficient and practically desirable, since a sudden spike in thrust could wear the rotors down, or send the quadrotor into a spin.

7 Extensions

Minimum deliverable: complete simulation study implementing all controllers, metrics and answer research question.

Extension: physical quadrotor deployment to evaluate sim-to-real transfer. Moreover we could expand project to something more complex, such as a fixed-wing UAV.

References

[1] L. Albrecht et al. *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. (MARL textbook; RL foundations and policy optimisation methods.)

[2] H. T. M. N. ElKholy, *Dynamic Modeling and Control of a Quadrotor Using Linear and Nonlinear Approaches*, Master's Thesis, Institution Name, Spring 2014.

[3] M. Rinaldi, S. Primatesta, and G. Guglieri, *A Comparative Study for Control of Quadrotor UAVs*, 2023.

[4] Newton-Euler equations (rigid body dynamics). https://en.wikipedia.org/wiki/Newton%E2%80%93Euler_equations

[5] Linear-quadratic regulator (LQR). https://en.wikipedia.org/wiki/Linear%E2%80%93quadratic_regulator